

Assignment 1

Nikhil Kapila, 11/08/2026

Fetches below and collates from HTML outputs downloaded from Colab runs.

Table 1: Decode tok/s, by runtime × model × prompt:

Runtime	TinyLlama short	TinyLlama long	GPT-2 short	GPT-2 long
CPU	5.41	4.22	23.21	15.03
T4 GPU	22.89	ERR	82.01	ERR
TPU v5e-1	0.48	OOM	0.58	0.42

Table 2: Prefill / Decode / TTFT (seconds)

Timing:

Runtime	Model	Prompt	Tok	Prefill (s)	Decode (s)	TTFT (s)	tok/s	Status
CPU	TinyLlama	short	33	1.58	9.06	1.79	5.41	OK
CPU	TinyLlama	long	169	4.96	11.62	5.16	4.22	OK
CPU	GPT-2	short	17	0.17	2.11	0.22	23.21	OK

CPU	GPT-2	long	14 5	0.77	3.26	0.83	15.0 3	OK
T4 GPU	TinyLlama	short	33	0.98	2.14	1.19	22.8 9	OK
T4 GPU	TinyLlama	long	—	—	—	—	—	ERR (profiler bug)
T4 GPU	GPT-2	short	17	0.08	0.60	0.14	82.0 1	OK
T4 GPU	GPT-2	long	—	—	—	—	—	ERR (profiler bug)
TPU v5e-1	TinyLlama	short	33	2.97	102.65	4.93	0.48	OK
TPU v5e-1	TinyLlama	long	—	—	—	—	—	ERR (OOM)
TPU v5e-1	GPT-2	short	17	1.68	84.85	4.67	0.58	OK
TPU v5e-1	GPT-2	long	14 5	4.63	115.81	6.96	0.42	OK

Cost (gpt-4o: \$2.50/1M in, \$10/1M out; output = 50 tok fixed):

Model	Prompt	Input tok	Output tok	Input \$	Output \$	Total \$	Output % of \$
TinyLlama	short	33	50	0.000082	0.000500	0.000583	85.8%
TinyLlama	long	169	50	0.000423	0.000500	0.000923	54.2%
GPT-2	short	17	50	0.000043	0.000500	0.000543	92.2%
GPT-2	long	145	50	0.000362	0.000500	0.000862	58.0%

Observation 1: In table 1, decode Tok/s table shows CPU TinyLLama at 5tps but CPU GPT at 23 tps.. Why is GPT2 faster at decode on same hardware?

I first compared the architecture and saw that GPT-2 has 124M parameters compared to TinyLlama having 1.1B params. Also, it's 12 layers vs 22 layers so it makes sense that GPT-2 decodes faster. Lesser layers = lesser weights to stream = less memory movement internally in the GPU = less streaming weights per token = faster tps!!

Also another thing to consider is the hidden dimension 768 vs 2048.

Att has 6 projs = Q,K,V, MLP up, MLP down.. Q,K,V is dx4d each, MLP up and down is dx4d each. So entire, weights is dx4d. Ratio of these yields tinnyllama having 7x more weights than each gpt-2 layer.. And I haven't even consider layers yet.

So, should be $(22 \times (2048 \times 2048)) / (12 \times (768 \times 768)) \sim 13.03$ ratio. Just showcasing the potential ratio of memory traffic per token.

Observation 2: TPU is slowest but from what I know it is a fast ML accelerator?

TPU is 0.48tps and 0.58tps compared to T4 22.89tps and 82tps on tinnyllama short and gpt2 short respectively. Also slower than CPU!!

I'm assuming this is something to do at the kernel level.. Or the inner workings of pytorch itself. Afaik, pytorch has a lot better support for cuda than anything else. I will have to look at the decode trace again to check and verify what its doing.

Observation 3: In table2, Prefill column for CPU TinyLlama short = 1.58 seconds, long =4.96 seconds but decode did not change much? 9.06s and 11.62 seconds.

Prefill processes all prompt tokens in one parallel forward pass so cost scales with more tokens. Decode in comparison is an autoregressive process since it only generates one token at a time. So longer prompt = more tokens to process = slower prefill. Also decode is done with kv cache so we dont have to re compute the old tokens again for new decode step!!

Decode doesn't change since it depends entirely on memory bandwidth and how fast things can be moved to the processor (at least for GPU). The key rule is our processor is fast! Memory unfortunately isn't!

The weight streaming per token is identical, same model and same hidden_dim.. Only thing longer prompt add is slightly bigger kv cache to read.

Observation 4: TTFT column, cpu tiny llama short is 1.79s and long is 5.16s..

prompt	prefill	decode	ttft
short	1.58	9.06	1.79
long	4.96	11.62	5.16

a) ttft tracks prefill numbers.. Decode is longer.. What is ttft measuring?

TTFT is measuring time to first token. Essentially when we're doing our first forward pass during prefill, we generate our first token that we need to decode on autoregressively later... furthermore decode happens afterwards!!

b) When prompt got longer, ttft jumped to 5.16. why?

When prompt got longer, ttft jumped further because everything is processed in parallel in the forward pass, higher tokens = more parallel tokens = more time to compute ?

User exp wise, user waits for longer to see first generation.. engineer will need to optimize prefill stage to make ttft lower

11/8/26

Matmul operations = $(2b-1)ac$ ($a \times b @ b \times c$)
 at high dims = $2abc$

In an LLM, there are many layers, each layer has: att + MLP sequentially.

Amounts to total 6 projections (proj) & 2 att ops.

Projection ops: (linear matmuls)

- | | | |
|----------------|------|--------------|
| 1) Q proj | MLP: | 5) up proj |
| 2) K proj | | 6) down proj |
| 3) V proj | | |
| 4) output proj | | |

Att core:

- 1) scores = $Q @ K^T$
- 2) out = softmax(scores) @ V

$\rightarrow (2b-1)ac$ approx.

op	$a \times b @ b \times c$	$2abc$	written as
Q proj	$n \times d @ d \times d$	$2ndd = 2nd^2$	$2nd^2$
scores	$n \times d @ d \times n$	$2ndn$	$2n^2d$

$n \rightarrow$ seq len (prompt len)
 $d \rightarrow$ hidden dim

$$\text{Ratio} = \frac{nd^2}{n^2d} = d/n$$

$d > n$, model wider than prompt len
 projections win

$n > d$, att dominates

Need 2 att ops to dominate/outweigh 6 proj's.